



SCRIPT PRESENTASI - SLIDE 31-40

Sequence Diagrams (lanjutan), Class, ERD, State Machine, Deployment

SLIDE 31: SD-09 - RECEIVE STOCK (PENERIMAAN STOK) (2 menit)

Narasi:

"Sequence Diagram ini adalah **SD-09 Receive Stock** - proses pencatatan penerimaan stok dari **SPBE** ke **Agen**."

[Point ke lifelines]

Ada **4 participants**:

- **Admin, PenerimaanPage, PenerimaanService**
- **penerimaan_stok, stock_histories** (databases)

[Trace flow menampilkan daftar]

Admin buka halaman Penerimaan Stok. PenerimaanService query data penerimaan berdasarkan bulan yang dipilih.

```
SELECT * FROM penerimaan_stok WHERE tanggal BETWEEN ? AND ?
```

Tampilkan daftar penerimaan yang ada.

[Trace flow tambah penerimaan baru]

Admin klik 'Tambah Penerimaan'. Sistem tampilkan form.

Admin input:

- **No. SO** (Sales Order dari SPBE)
- **No. LO** (Loading Order)
- **Nama material** (jenis LPG)
- **Quantity** dalam pcs dan kg
- **Tanggal** penerimaan
- **Sumber** (SPBE mana)

Lalu klik Simpan.

[Point ke proses sistem]

PenerimaanService:

1. **validateInput()** - validasi data
2. **INSERT INTO penerimaan_stok** - simpan data penerimaan

3. `mapMaterialToLpgType()` - mapping nama material ke lpg_type

4. **INSERT INTO stock_histories** dengan movement_type='MASUK'

[Point ke note - stock_histories]

Perhatikan ada **note**: stock_histories mencatat pergerakan stok **AGEN** (bukan pangkalan). Ini untuk tracking stok masuk dari SPBE.

Diagram ini menunjukkan **upstream supply chain** - penerimaan dari supplier ke agen."

SLIDE 32: SD-10 - RECORD DISTRIBUTION (PENYALURAN) (2 menit)

Narasi:

"Sequence Diagram ini adalah **SD-10 Record Distribution** - proses pencatatan **penyaluran harian** ke pangkalan. penyaluran ini singkatnya adalah pengiriman LPG dari agen ke pangkalan. yg nantinya akan ter record datanya di database.

[Point ke lifelines]

Ada **5 participants**:

- Admin, PenyaluranPage, PenyaluranService
- pangkalans, penyaluran_harian, stock_histories

[Trace flow menampilkan grid]

Admin buka halaman Penyaluran, pilih bulan dan tipe LPG.

lalu di PenyaluranService query:

- `SELECT * FROM pangkalans WHERE is_active = true` - ini artinya pangkalan aktif
- `SELECT * FROM penyaluran_harian WHERE bulan = ? AND lpg_type = ?` - ini artinya data penyaluran existing untuk bulan tersebut
- `buildGrid(pangkalans, data)` - ini artinya build data dalam format grid (pangkalan × tanggal)

Tampilkan grid yang bisa di-edit.

[Trace flow input penyaluran]

Admin klik cell untuk input, masukkan:

- **Jumlah normal** - penyaluran reguler
- **Jumlah fakultatif** - penyaluran tambahan
- **Tipe pembayaran**

Lalu klik Simpan.

[Point ke LOOP fragment - bulk update]

Ada **LOOP fragment** untuk setiap item yang diinput.

Untuk setiap item:

- Cek apakah data sudah ada di penyaluran_harian

[Point ke ALT fragment]

Ada **ALT fragment**:

- Jika **data sudah ada** → UPDATE
- Jika **data belum ada** → INSERT

Kemudian **INSERT stock_histories** dengan movement_type='KELUAR' untuk mencatat stok keluar dari agen.

Diagram ini menunjukkan **bulk data entry** untuk efisiensi input harian."

SLIDE 33: SD-19 - GENERATE & EXPORT REPORT (PANGKALAN) (2.5 menit)

Narasi:

"Sequence Diagram ini adalah **SD-19 Generate & Export Report** untuk aktor **Pangkalan** - menampilkan fitur laporan dan export.

[Point ke lifelines]

Ada **5 participants** (participants itu objek-objek yang berinteraksi dalam sequence):

- **Pangkalan** (aktor) - user yang mengakses
- **LaporanPage** (boundary) - halaman UI
- **LaporanService** (control) - logic di backend
- **consumer_orders, expenses, lpg_prices** (entity) - tabel database

[Trace flow load dashboard]

Pangkalan buka halaman Laporan. LaporanService query data dari database:

- **consumer_orders** - data penjualan
- **expenses** - data pengeluaran
- **lpg_prices** - harga LPG

Kemudian kalkulasi (menghitung):

- **calculateTotalPenjualan()** - total pendapatan
- **calculateTotalPengeluaran()** - total pengeluaran
- **calculateLabaBersih()** - pendapatan dikurangi pengeluaran

[Trace flow export]

Pangkalan bisa export ke Excel atau PDF. Sistem generate file dengan format yang rapi lalu trigger download.

Diagram ini menunjukkan fitur **reporting** untuk pangkalan."

SLIDE 34: SD-02 - LOGOUT (1 menit)

Narasi:

"Sequence Diagram ini adalah **SD-02 Logout** - simple tapi penting untuk keamanan.

[Trace flow logout]

User klik 'Logout'. Sistem:

- 1. **Hapus token dari browser** - JWT token dihapus dari penyimpanan lokal
- 2. **Update database** - session_id di-set NULL

Kenapa session_id di-NULL-kan? Karena sistem menggunakan **Single-Session Login** - user hanya bisa login di 1 device. Dengan menghapus session_id, token lama otomatis tidak berlaku lagi.

Terakhir, redirect ke halaman Login."

SLIDE 35: CLASS DIAGRAM (5 menit)

Narasi:

"Sekarang masuk ke **Class Diagram**. Class Diagram ini menggambarkan **struktur class atau objek** dalam sistem SIM4LON beserta hubungan antar class tersebut.

[Point ke title]

Secara keseluruhan, SIM4LON memiliki **23 Classes** yang dikelompokkan dalam **6 Packages** dan menggunakan **7 Enumerations** sebagai tipe data tetap.

BAGIAN 1: ENUMERATIONS

[Point ke package Enumerations - kuning]

Pertama saya jelaskan tentang **Enumerations** atau **Enum**. Enum adalah tipe data yang nilainya sudah ditentukan dan tidak bisa diisi sembarangan. Ini penting untuk menjaga **konsistensi data** di database.

SIM4LON menggunakan **7 enum**:

 Referensi Enum:

Enum	Nilai	Fungsi
user_role	ADMIN, OPERATOR, PANGKALAN	3 level akses berbeda
status_pesanan	DRAFT → MENUNGGU PEMBAYARAN → DIPROSES → SIAP_KIRIM → DIKIRIM → SELESAI / BATAL	State machine order
lpg_type	kg3, kg5, kg12, kg50, gr220	Jenis tabung LPG
lpg_category	SUBSIDI, NON_SUBSIDI	Penentuan PPN (0% vs 12%)

Enum	Nilai	Fungsi
<code>payment_method</code>	TUNAI, TRANSFER	Metode pembayaran
<code>stock_movement_type</code>	MASUK, KELUAR	Arah pergerakan stok
<code>consumer_type</code>	RUMAH_TANGGA, WARUNG	Tipe konsumen untuk verifikasi subsidi

Contoh penggunaan, kalau user mau input status pesanan, dia hanya bisa pilih dari 7 status yang sudah ditentukan, tidak bisa isi status sembarangan seperti "PENDING" atau "PROCESS" karena sistem akan tolak.

BAGIAN 2: PACKAGES DAN CLASSES

[Point ke packages]

Selanjutnya tentang pengelompokan class. Class-class di SIM4LON saya kelompokkan dalam **6 packages** berdasarkan fungsinya. Package ini seperti folder yang mengelompokkan class-class sejenis agar kode lebih terorganisir.

Referensi Package:

Package	Warna	Classes	Fungsi
Master Data	Hijau	users, agen, pangkalans, drivers, lpg_products, company_profile	Data dasar sistem
Order Management	Biru	orders, order_items, timeline_tracks, invoices	Pengelolaan pesanan
Payment	Oranye	order_payment_details, payment_records	Pembayaran
Stock Management	Pink	stock_histories, penerimaan_stok, perencanaan_harian, penyaluran_harian	Stok agen
Pangkalan SAAS	Ungu	consumers, consumer_orders, pangkalan_stocks, lpg_prices, expenses, agen_orders	Operasional multi-tenant
Audit & Logging	Abu	activity_logs	Jejak audit

Untuk **Master Data** di warna hijau, berisi data-data dasar seperti users untuk login, agen untuk data distributor, pangkalans untuk data pangkalan, dan drivers untuk data pengemudi.

Yang menarik adalah class **company_profile** - ini adalah **singleton**, artinya hanya ada 1 record di database karena profil perusahaan memang cuma satu.

Untuk **Order Management** di warna biru, berisi orders sebagai pesanan utama, order_items untuk detail item, timeline_tracks untuk riwayat status, dan invoices untuk dokumen tagihan.

BAGIAN 3: RELATIONSHIPS

[Point ke garis-garis antar class]

Bagian yang paling penting adalah **Relationships** atau hubungan antar class. Dalam UML, ada **4 jenis hubungan** yang saya gunakan di SIM4LON:

3.1. ASSOCIATION — Garis biasa

Yang pertama **Association**. Association adalah hubungan paling umum, dimana kedua objek **bisa hidup mandiri** - tidak saling bergantung. Contohnya user dengan pangkalan, meskipun pangkalan dihapus, data user-nya tetap ada.

 **Contoh Association:**

Relasi	Penjelasan
users ↔ pangkalans	User PANGKALAN terhubung ke 1 pangkalan
pangkalans ↔ agen	Banyak pangkalan di bawah 1 agen
orders ↔ drivers	Order bisa punya driver (opsional)

Ciri khasnya: FK dengan **onDelete: NoAction** - parent dihapus, child tetap ada.

3.2. AGGREGATION — Diamond kosong ◇

Yang kedua **Aggregation**, ditandai dengan **diamond kosong**. Aggregation adalah hubungan "memiliki" dimana child **bisa ada tanpa parent**. Contohnya pangkalan memiliki konsumen, tapi konsumen bukan bagian integral dari pangkalan dan bisa dipindah ke pangkalan lain.

 **Contoh Aggregation:**

Relasi	Penjelasan
pangkalans ◇— consumers	Pangkalan punya banyak konsumen
pangkalans ◇— pangkalan_stocks	Pangkalan punya data stok per jenis LPG
pangkalans ◇— expenses	Pangkalan punya record pengeluaran
orders ◇— invoices	Order bisa generate banyak invoice

Ciri khasnya: FK dengan **onDelete: NoAction** - parent dihapus, child tetap ada untuk audit trail.

3.3. COMPOSITION — Diamond hitam ◆

Yang ketiga **Composition**, ditandai dengan **diamond hitam**. Composition adalah hubungan paling kuat dimana child **tidak bisa ada tanpa parent**. Kalau parent dihapus, child otomatis ikut terhapus.

Contoh Composition:

Relasi	Penjelasan
<code>orders</code> ◆— <code>order_items</code>	Order HARUS punya minimal 1 item, hapus order = hapus items
<code>orders</code> ◆— <code>timeline_tracks</code>	Timeline adalah bagian integral dari order
<code>orders</code> ◆— <code>order_payment_details</code>	Summary pembayaran per order (one-to-one)

Ciri khasnya: FK dengan `onDelete: Cascade` - parent dihapus, child ikut terhapus.

Jadi kalau kita **DELETE order ORD-0050**, maka `order_items`, `timeline_tracks`, dan `payment_details` yang terkait **otomatis terhapus** oleh database.

3.4. DEPENDENCY — Garis putus-putus ···→

Yang terakhir **Dependency**, ditandai dengan **garis putus-putus**. Dependency adalah ketika class hanya **menggunakan** class lain sebagai tipe data, tapi tidak menyimpan referensi permanen. Di SIM4LON, ini dipakai untuk relasi ke **Enum**.

Contoh Dependency:

Relasi	Penjelasan
<code>orders</code> ..> <code>status_pesanan</code>	Order menggunakan enum untuk status
<code>order_items</code> ..> <code>lpg_type</code>	Item menggunakan enum untuk jenis LPG
<code>users</code> ..> <code>user_role</code>	User menggunakan enum untuk role

Ciri khasnya: Tidak ada FK di database, hanya tipe data column yang mengacu ke enum.

BAGIAN 4: MULTIPLICITY (Kardinalitas)

[Jelaskan angka di ujung garis]

Notasi	Arti	Contoh di SIM4LON
1	Tepat 1, wajib	Order HARUS punya 1 pangkalan
0..1	Opsional, maksimal 1	Order BISA punya 1 driver (atau tidak ada)
0..*	Nol atau banyak	Pangkalan BISA punya 0, 1, atau banyak order
1..*	Minimal 1, bisa banyak	Order HARUS punya minimal 1 item

BAGIAN 5: RINGKASAN RELASI SIM4LON

Tabel Lengkap Semua Relasi:

Parent Class	Child Class	Type Relasi	Multiplicity	Cascade Delete?
agen	pangkalans	Aggregation	1 : 0..*	✗ No
pangkalans	users	Association	1 : 0..*	✗ No
pangkalans	orders	Aggregation	1 : 0..*	✗ No
pangkalans	consumers	Aggregation	1 : 0..*	✗ No
pangkalans	consumer_orders	Aggregation	1 : 0..*	✗ No
pangkalans	pangkalan_stocks	Aggregation	1 : 0..*	✗ No
pangkalans	lpg_prices	Aggregation	1 : 0..*	☑ Yes
pangkalans	expenses	Aggregation	1 : 0..*	✗ No
pangkalans	penyaluran_harian	Aggregation	1 : 0..*	✗ No
pangkalans	perencanaan_harian	Aggregation	1 : 0..*	✗ No
pangkalans	agen_orders	Aggregation	1 : 0..*	✗ No
orders	order_items	Composition	1 : 1..*	☑ Yes
orders	timeline_tracks	Composition	1 : 0..*	☑ Yes
orders	order_payment_details	Composition	1 : 0..1	☑ Yes
orders	invoices	Aggregation	1 : 0..*	✗ No
orders	payment_records	Aggregation	1 : 0..*	✗ No
orders	activity_logs	Aggregation	1 : 0..*	✗ No
orders	drivers	Association	0..* : 0..1	✗ No
lpg_products	stock_histories	Aggregation	1 : 0..*	✗ No
users	stock_histories	Association	1 : 0..*	✗ No
users	payment_records	Association	1 : 0..*	✗ No
users	activity_logs	Association	1 : 0..*	✗ No
consumers	consumer_orders	Association	0..1 : 0..*	✗ No
invoices	payment_records	Association	0..1 : 0..*	✗ No
agen	agen_orders	Association	0..1 : 0..*	✗ No

KESIMPULAN

Design Pattern yang digunakan:

1. **Cascade Delete** hanya pada relasi **Composition** (order → items/timeline/payment)
 - Alasan: Data child tidak bermakna tanpa parent

2. **NoAction** pada relasi **Aggregation** (pangkalan → consumers/stocks)

- Alasan: Data tetap diperlukan untuk audit trail meski parent dihapus

3. **Soft Delete** (**deleted_at**) pada master data (users, agen, pangkalans, drivers)

- Alasan: Tidak menghapus fisik, hanya menandai sebagai terhapus

Dengan **23 classes terstruktur** dan relasi yang tepat, sistem ini:

- **Modular** - mudah dipahami per package
- **Maintainable** - mudah diupdate dengan aturan cascade yang jelas
- **Data Integrity** - integritas data terjaga dengan foreign key"

SLIDE 36: ERD (ENTITY RELATIONSHIP DIAGRAM) (5 menit)

Narasi:

"Baik, sekarang kita lanjut ke **ERD** atau **Entity Relationship Diagram**. Kalau Class Diagram tadi menggambarkan struktur dari sisi pemrograman, ERD ini menggambarkan **struktur tabel di database** - bagaimana data disimpan dan saling terhubung.

[Point ke title]

Database SIM4LON menggunakan **PostgreSQL 15** dengan **ORM Prisma** dan total **23 tabel**.

BAGIAN 1: NOTASI ERD

[Point ke legend]

Pertama saya jelaskan notasi yang digunakan. Ada **3 jenis key** di database:

Jenis Key:

Simbol	Nama	Penjelasan
 PK	Primary Key	ID unik setiap record, pakai UUID
 FK	Foreign Key	Penghubung ke tabel lain
 UK	Unique Key	Kolom yang nilainya harus unik

Kenapa SIM4LON pakai **UUID** sebagai Primary Key dan bukan auto-increment biasa? Ada 3 alasan:

1. Lebih aman - tidak bisa ditebak urutannya
2. Cocok untuk distributed system, distributed sistem ini yaitu sistem yang dijalankan di beberapa server
3. Bisa di-generate di frontend tanpa tunggu database

BAGIAN 2: TIPE DATA

[Point ke kolom-kolom tabel]

Selanjutnya tentang tipe data yang digunakan. Ini penting untuk memahami kenapa saya pilih tipe data tertentu.

📄 Tipe Data Utama:

Tipe	Penggunaan	Contoh
UUID	Primary/Foreign Key	<code>id, pangkalan_id</code>
varchar(n)	Teks pendek	<code>name, email, code</code>
text	Teks panjang	<code>address, note</code>
decimal(15,2)	Uang	<code>total_amount, price</code>
boolean	True/False	<code>is_active, is_paid</code>
timestamptz	Waktu + timezone	<code>created_at, updated_at</code>
enum	Pilihan tetap	<code>role, status</code>

Yang menarik adalah tipe **decimal(15,2)** untuk uang. Kenapa tidak pakai float? Karena float punya masalah presisi - coba di kalkulator, $0.1 + 0.2$ kadang hasilnya 0.30000000001 . Untuk uang, itu tidak bisa diterima. Dengan decimal 15,2, kita bisa simpan sampai Rp 9,9 triliun dengan presisi 2 desimal.

BAGIAN 3: CARDINALITY (Kardinalitas)**[Point ke garis-garis relasi antar tabel]**

Bagian yang paling penting di ERD adalah **Cardinality** atau Kardinalitas. Cardinality menunjukkan **berapa banyak record** di satu tabel yang bisa terhubung ke tabel lain.

Ada **4 jenis cardinality** yang digunakan di SIM4LON:

📄 Jenis Cardinality:

Notasi ERD	Nama	Arti	Contoh
1 -- 0..*	One to Many (Optional)	1 record parent bisa punya 0 atau banyak child	1 Agen bisa punya 0, 1, atau 50 Pangkalan
1 -- 1..*	One to Many (Required)	1 record parent HARUS punya minimal 1 child	1 Order HARUS punya minimal 1 Item
1 -- 0..1	One to One (Optional)	1 record parent bisa punya maksimal 1 child	1 Order BISA punya 1 Payment Details (atau tidak)
1 -- 1	One to One (Required)	1 record parent HARUS punya tepat 1 child	Tidak ada di SIM4LON

Sekarang saya jelaskan semua **27 relasi** yang ada di ERD SIM4LON:

3.1. RELASI 1 -- 0.. (One to Many, Optional)*

Ini adalah tipe relasi paling umum - 1 parent bisa punya **nol atau banyak** child.

 Semua Relasi 1 -- 0.. .*

From (Parent)	To (Child)	Penjelasan
agen	pangkalans	1 Agen bisa memiliki banyak Pangkalan
pangkalans	users	1 Pangkalan bisa punya banyak User PANGKALAN role
pangkalans	orders	1 Pangkalan bisa punya banyak Pesanan
pangkalans	consumers	1 Pangkalan bisa punya banyak Konsumen
pangkalans	consumer_orders	1 Pangkalan bisa punya banyak Penjualan
pangkalans	lpg_prices	1 Pangkalan bisa set banyak Harga per tipe LPG
pangkalans	pangkalan_stocks	1 Pangkalan bisa punya banyak record Stok
pangkalans	pangkalan_stock_movements	1 Pangkalan bisa punya banyak History Stok
pangkalans	expenses	1 Pangkalan bisa punya banyak Pengeluaran
pangkalans	penyaluran_harian	1 Pangkalan bisa punya banyak record Penyaluran
pangkalans	perencanaan_harian	1 Pangkalan bisa punya banyak record Perencanaan
pangkalans	agen_orders	1 Pangkalan bisa punya banyak Order ke Agen
drivers	orders	1 Driver bisa mengirim banyak Pesanan
orders	timeline_tracks	1 Order bisa punya banyak Timeline
orders	invoices	1 Order bisa generate banyak Invoice
orders	payment_records	1 Order bisa punya banyak Record Pembayaran
orders	activity_logs	1 Order bisa punya banyak Activity Log
invoices	payment_records	1 Invoice bisa punya banyak Record Pembayaran
users	payment_records	1 User bisa mencatat banyak Pembayaran
users	stock_histories	1 User bisa mencatat banyak History Stok
users	activity_logs	1 User bisa generate banyak Activity Log
lpg_products	stock_histories	1 Product bisa punya banyak History Stok
consumers	consumer_orders	1 Konsumen bisa punya banyak Pesanan
agen	agen_orders	1 Agen bisa menerima banyak Order dari Pangkalan

3.2. RELASI 1 -- 1.. (One to Many, Required)*

Tipe relasi dimana child **WAJIB ada** minimal 1.

 Relasi 1 -- 1..*

From (Parent)	To (Child)	Penjelasan	Konsekuensi
orders	order_items	1 Order HARUS punya minimal 1 Item	Order tanpa item = TIDAK VALID

Kenapa 1..? Karena secara bisnis, pesanan tanpa item tidak masuk akal. Di backend, kita validasi ini sebelum create order.*

3.3. RELASI 1 -- 0..1 (One to One, Optional)

Tipe relasi dimana 1 parent bisa punya **maksimal 1** child (opsional).

 Relasi 1 -- 0..1 :

From (Parent)	To (Child)	Penjelasan	Implementasi
orders	order_payment_details	1 Order bisa punya 1 Payment Summary	FK <code>order_id</code> di-set UNIQUE

Kenapa One-to-One? Karena `order_payment_details` adalah **summary pembayaran** per order - tidak perlu lebih dari 1 record. Bedanya dengan menyimpan langsung di tabel `orders`:

- **Separation of Concerns** - data pembayaran terpisah dari data order
- **Nullable** - order bisa ada tanpa payment details (saat masih DRAFT)

KESIMPULAN CARDINALITY

Tipe	Jumlah Relasi	Contoh Utama
1 -- 0..*	24 relasi	pangkalans → orders, consumers
1 -- 1..*	1 relasi	orders → order_items
1 -- 0..1	1 relasi	orders → order_payment_details
Total	26 relasi	

Yang paling sering digunakan adalah **1 -- 0..*** karena fleksibel - bisa ada atau tidak ada child.

BAGIAN 4: DESIGN PATTERNS

[Jelaskan pattern yang digunakan]

Terakhir saya jelaskan beberapa design pattern yang diimplementasikan:

📋 Pattern yang Digunakan:

Pattern	Implementasi	Tabel
Soft Delete	<code>deleted_at</code> instead of DELETE	users, agen, pangkalans, drivers, orders
Audit Trail	<code>created_at</code> , <code>updated_at</code> otomatis	Semua tabel
Cascade Delete	<code>onDelete: Cascade</code>	order_items, timeline_tracks
Normalisasi 3NF	Tidak ada transitive dependency	Semua tabel

Soft Delete artinya data tidak benar-benar dihapus dari database, hanya ditandai dengan timestamp di kolom `deleted_at`. Ini penting untuk audit - kalau ada pertanyaan tentang data historis, kita masih bisa melacaknya.

Dengan **23 tabel terstruktur** dan design pattern yang tepat, database SIM4LON ini:

- **Scalable** - siap untuk ratusan pangkalan
- **Secure** - multi-tenant dengan isolasi data
- **Auditable** - semua perubahan tercatat"

SLIDE 37: STATE MACHINE - ORDER STATUS (SM-01) (2 menit)

Narasi:

"Sekarang **State Machine**. State Machine adalah diagram yang menunjukkan **perubahan status** suatu objek.

SM-01 ini untuk **Status Pesanan**.

[Point ke states]

Ada **7 status**:

1. **DRAFT** - pesanan baru dibuat
2. **MENUNGGU PEMBAYARAN** - menunggu bayar
3. **DIPROSES** - pembayaran diterima, barang disiapkan
4. **SIAP_KIRIM** - siap dikirim
5. **DIKIRIM** - dalam perjalanan
6. **SELESAI** - pesanan selesai, **stok pangkalan otomatis bertambah**
7. **BATAL** - pesanan dibatalkan

[Point ke transitions]

Panah menunjukkan **transisi** (perpindahan status). Contoh:

- MENUNGGU PEMBAYARAN → DIPROSES hanya jika sudah bayar
- SIAP_KIRIM → DIKIRIM hanya jika sudah ada driver

BATAL bisa dari status manapun kecuali SELESAI.

State Machine ini **benar-benar diimplementasikan** di backend - kalau transisi tidak valid, sistem akan tolak."

SLIDE 38: STATE MACHINE - AGEN ORDER STATUS (SM-02) (1.5 menit)

Narasi:

"SM-02 untuk **Order dari Pangkalan ke Agen** (B2B - business to business).

[Point ke states]

Ada **4 status**:

1. **PENDING** - pangkalan pesan, menunggu konfirmasi agen
2. **DIKIRIM** - agen sudah kirim
3. **DITERIMA** - pangkalan terima, stok bertambah
4. **DITOLAK** - ditolak (misal stok agen habis)

[Point ke partial delivery]

Menariknya, qty_received (jumlah diterima) bisa berbeda dengan qty_ordered (jumlah dipesan). Ini untuk kasus **partial delivery** - misal pesan 50, tapi yang kondisi baik cuma 48."

SLIDE 39: STATE MACHINE - USER SESSION (SM-03) (1 menit)

Narasi:

"SM-03 untuk **Session User** - fitur **Single Session Login**.

[Point ke states]

Ada **4 status**:

1. **LOGGED_OUT** - belum login
2. **LOGGED_IN** - sedang login aktif
3. **SESSION_EXPIRED** - token kadaluarsa
4. **KICKED_OUT** - terlempar karena login di device lain

[Fitur Single Session]

Jika user login di HP, lalu login lagi di laptop, session di HP **otomatis logout**. Ini mencegah **sharing akun** dan meningkatkan keamanan."

SLIDE 40: DEPLOYMENT DIAGRAM (2.5 menit)

Narasi:

"Terakhir **Deployment Diagram** - menggambarkan **arsitektur sistem di production** (server yang sudah online).

[Penjelasan diagram]

Deployment Diagram menunjukkan **dimana aplikasi di-deploy** dan **bagaimana komponen saling terhubung**.

[Point ke komponen]

Komponen SIM4LON:

- 1. **Vercel** - hosting frontend (website yang user akses)
 - Astro 5 + React 18 untuk UI
 - Tailwind CSS untuk styling
- 2. **Railway** - hosting backend (server API)
 - NestJS 11 untuk REST API
 - Prisma ORM untuk query database
- 3. **PostgreSQL** - database (penyimpanan data)
 - 23 tabel, backup harian
- 4. **Supabase Storage** - penyimpanan file
 - Foto profil, bukti transfer
- 5. **Google Gemini AI** - untuk fitur Voice Order

[Point ke koneksi]

Panah menunjukkan **protokol komunikasi**:

- HTTPS untuk akses web (aman)
- TCP:5432 untuk koneksi database

[Kesimpulan]

Aplikasi sudah **live** di **sim4lon.vercel.app**."

 TIMING SLIDE 31-40:

Slide	Diagram	Durasi	Kumulatif
31	SD-09 Receive Stock	2m	2:00
32	SD-10 Record Distribution	2m	4:00
33	SD-19 Export Report	2.5m	6:30
34	SD-02 Logout	1m	7:30
35	Class Diagram	3.5m	11:00
36	ERD	3m	14:00

Slide	Diagram	Durasi	Kumulatif
37	SM-01 Order Status	2.5m	16:30
38	SM-02 Agen Order	2m	18:30
39	SM-03 User Session	1.5m	20:00
40	Deployment Diagram	3m	23:00

Total Slide 31-40: ~23 menit

Running Total (1-40): ~56 menit ☒